```
...     print key, monthList[key]
Feb 2
Aug 8
Jan 1
Dec 12
Oct 10
Mar 3
Sep 9
May 5
Jun 6
Jul 7
Apr 4
Nov 11
```

Note: The order is based on the hash ordering of objects in the dictionary and, hence, the random output.

List comprehensions

List comprehensions are easy ways of generating lists. Consider the following code to generate a list containing numbers from 1 to 25:

```
>>> alist=[]
>>> for i in range(1, 26):
...     alist.append(i)
>>> alist
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
19, 20, 21, 22, 23, 24, 25]
```

An alternative, functional approach to the same would be as follows:

```
>>> alist=[i for i in range(1, 26)]
>>> alist
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,
19, 20, 21, 22, 23, 24, 25]
```

Notice that this code is more intuitive and easy to understand—it is close to plain English!

Another example—how to strip white-space from a list of strings:

```
>>> alist=['       This is line 1         \n', 'This is line 2
\n', '', '               This is line 3\n']
>>> [line.strip() for line in alist]
['This is line 1', 'This is line 2', '', 'This is line 3']
```

Here's a modification of the above, which tests if a string is a null string, and doesn't print if that is the case.

```
>>> [line.strip() for line in alist
...             if line != '']
['This is line 1', 'This is line 2', 'This is line 3']
>>>
```

The actual syntax of list comprehension is as follows:

```
[ expression for expr in sequence1
             if condition1
             for expr2 in sequence2
             if condition2
             for expr3 in sequence3 ...
             if condition3
             for exprN in sequenceN
             if conditionN ]
```

The equivalent Python code for the above is shown below:

```
for expr1 in sequence1:
     if not (condition1):
       continue    # Skip this element
     for expr2 in sequence2:
       if not (condition2):
           continue    # Skip this element
       ...
       for exprN in sequenceN:
           if not (conditionN):
               continue    # Skip this element
           # Output the value of the expression.
```

While creating tuples in list comprehensions, it must be surrounded by parentheses. In the code given below, the first list comprehension results in an error, while the second one is correct:

```
>>> seq1=[1,2,3]
>>> seq2=['a','b','c']
>>> [x, y for x in seq1 for y in seq2]
  File "<stdin>", line 1
    [x, y for x in seq1 for y in seq2]
          ^
SyntaxError: invalid syntax
>>> [(x, y) for x in seq1 for y in seq2]
[(1, 'a'), (1, 'b'), (1, 'c'), (2, 'a'), (2, 'b'), (2, 'c'), (3,
'a'), (3, 'b'), (3, 'c')]
```

Map

Map is a built-in function that takes a function and a list as an argument, applies the function to each element of the list, and returns the output. For instance, *map(f, iterA, iterB, ...)* returns a list containing *f(iterA[0], iterB[0])*, *f(iterA[1], iterB[1])*, *f(iterA[2], iterB[2])*, ... For example, the following code finds the square of each element in a list: